

Semantic Analysis

Chapter 5

The Limitation of CFG

- Consider the following grammar and programs
 - Does the corresponding parser accept the programs?

```
S -> Decl Stmt  
Decl -> Type id ; | Decl; Decl  
Type -> string | int  
Stmt -> Stmt; Stmt |  
        id = Exp | ...  
Exp -> Exp + Exp | id | num | ...
```

```
string x;  
int z;  
  
x = "hello world";  
z = x + 1
```

- Many things cannot be decided by syntax analysis
 - Does the declared type of a variable match the value type?
 - Is an array access out of bound?
 - Where should a variable be stored (heap, stack, ...)
 - ...
- The answers to these questions depend on values instead of syntax.

The Semantic Analysis Phase

The *semantic analysis* phase of a compiler:

- Determine some static properties of a program via AST, including
 - Scope and visibility of names
 - every variable is declared before use
 - Types of variables, functions, and expressions
 - every expression has a proper type
 - function calls conform to definitions
 - ...
- Translate the AST into a simpler representation suitable for generating machine code (i.e., intermediate code, IR)
 - Section 7 of the Tiger book

Outline

- **Symbol Tables**
- Symbols in the Tiger Compiler
- Type Checking

Symbol Tables

- The semantic analysis phase is characterized by the maintenance of **symbol tables** (or **environments**) mapping identifiers to their types and locations.
- **Binding**: given a meaning to a symbol, denoted by $\mapsto$

- e.g.,

Name/Symbol	Meaning/Attribute
type identifier	type (e.g., int, string)
variable identifier	type, value, access info, ...
function identifier	args.& result types, ...

- **Environment**: a set of bindings
 - e.g., $\sigma_0 = \{g \mapsto \text{string}, a \mapsto \text{int}\}$
- **Symbol table**: the implementation of environment

The **semantic analysis** phase traverses the abstract syntax tree (AST) in certain order while maintaining symbol tables.

Motivating Example of Symbol Tables

- Suppose the environment is σ_0 before the first line

```
1 function f(a:int,b:int,c:int)=  
2   (print_int (a+c);  
3   let var j:= a+b  
4       var a:= "hello"  
5   in print(a); print_int(j)  
6 end;  
7 print_int(b)  
8 )
```

$\sigma_1 = \sigma_0 + \{a \mapsto \text{int}, b \mapsto \text{int}, c \mapsto \text{int}\}$

$\sigma_2 = \sigma_1 + \{j \mapsto \text{int}\}$

- Declarations of identifiers: IDs are bound to “meanings” in the symbol tables.
- Use of Identifiers: look up IDs in the symbol tables.
 - In line 3, the identifiers *a* and *b* can be looked up in σ_1
- In line 3, how does the compiler know the type of identifier *j*?

Motivating Example of Symbol Tables

- Suppose the environment is σ_0 before the first line

```
1 function f(a:int,b:int,c:int)=  
2   (print_int (a+c);  
3   let var j:= a+b  
4     var a:= "hello"  
5   in print(a); print_int(j)  
6 end;  
7 print_int(b)  
8 )
```

$\sigma_1 = \sigma_0 + \{a \mapsto \text{int}, b \mapsto \text{int}, c \mapsto \text{int}\}$

$\sigma_2 = \sigma_1 + \{j \mapsto \text{int}\}$

$\sigma_3 = \sigma_2 + \{a \mapsto \text{string}\}$

- Consider Line 4:
 - σ_2 contains $a \mapsto \text{int}$, what is the binding of a in σ_3 ?
 - $a \mapsto \text{int}$ overrides the binding of a in σ_2
- Bindings in the right-hand table **override** those in the left.
- $X + Y$ for tables is not the same as $Y + X$.

Motivating Example of Symbol Tables

- Each local variable has a *scope* in which it is visible

```
1 function f(a:int,b:int,c:int)=  
2   (print_int (a+c);  
3   let var j:= a+b  
4     var a:= "hello"  
5   in print(a); print_int(j)  
6 end;  
7 print_int(b)  
8 )
```

$\sigma_1 = \sigma_0 + \{a \mapsto \text{int}, b \mapsto \text{int}, c \mapsto \text{int}\}$

$\sigma_2 = \sigma_1 + \{j \mapsto \text{int}\}$

$\sigma_3 = \sigma_2 + \{a \mapsto \text{string}\}$

“Discard” σ_2 and σ_3

Look up b in σ_1

Discard σ_1 , go back to σ_0

- For `let D in E end`, the scope of each identifier declared in `D` starts from its declaration and ends after `end`.
- As the semantic analyzer **reaches the end of each scope**, the identifier bindings local to that scope are **discarded**
 - In line 6, σ_3 is discarded and σ_1 is restored.

The Interface of Symbol Tables

- **insert**: inserts a binding into the table
- **lookup**: looks up a name and obtain the information bound to the name
- **beginScope**: enters a new scope
- **endScope**: exits a scope, and restores the symbol table to the state before entering the scope

Multiple Symbol Tables

- In some languages there can be several active environments at once
 - Each module, or class, or record in the program has a **symbol table** σ of its own.

```
package M;  
class E {  
    static int a = 5;  
}  
class N {  
    static int b = 10;  
    static int a = E.a + b;  
}  
class D {  
    static int d = E.a + N.a;  
}
```

Java

$\sigma1 = \{ a \mapsto \textit{int} \}$

$\sigma2 = \{ E \mapsto \sigma1 \}$

$\sigma3 = \{ b \mapsto \textit{int}, a \mapsto \textit{int} \}$

$\sigma4 = \{ N \mapsto \sigma3 \}$

$\sigma5 = \{ d \mapsto \textit{int} \}$

$\sigma6 = \{ D \mapsto \sigma5 \}$

$\sigma7 = \sigma2 + \sigma4 + \sigma6$

In Java, forward reference is allowed. so E , N , and D are all compiled in the environment $\sigma7$. The result of the analysis is $\{M \mapsto \sigma7\}$.

Multiple Symbol Tables

```
structure M = struct
  structure E = struct
    val a = 5;
  end
  structure N = struct
    val b = 10
    val a = E.a + b
  end
  structure D = struct
    val d = E.a + N.a
  end
end
```

ML

```
 $\sigma_1 = \{ a \mapsto \text{int} \}$   
 $\sigma_2 = \{ E \mapsto \sigma_1 \}$   
 $\sigma_3 = \{ b \mapsto \text{int}, a \mapsto \text{int} \}$   
 $\sigma_4 = \{ N \mapsto \sigma_3 \}$   
 $\sigma_5 = \{ d \mapsto \text{int} \}$   
 $\sigma_6 = \{ D \mapsto \sigma_5 \}$   
 $\sigma_7 = \sigma_2 + \sigma_4 + \sigma_6$ 
```

N is compiled using environment $\sigma_0 + \sigma_2$. **D** is compiled using $\sigma_0 + \sigma_2 + \sigma_4$.

The result of the analysis is also $\{M \mapsto \sigma_7\}$.

- Each class/structure has its own symbol table.

Implementing Symbol Tables

Imperative Style

- Modify σ_1 until it becomes σ_2
- While σ_2 exists, we cannot look things up in σ_1
- When we are done with σ_2 , we can undo the modification to get σ_1 back again
 - a single global environment σ + an **undo stack**

Functional Style

- Keep σ_1 in pristine condition while creating σ_2 and σ_3
- Easy to restore σ_1

Either style can be used.

Efficient Imperative Symbol Tables

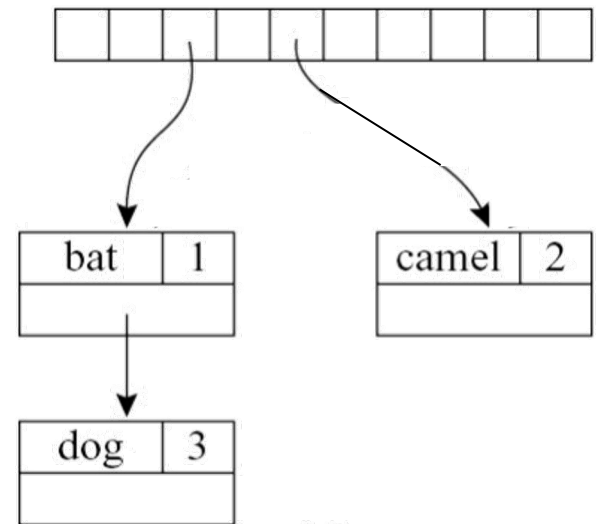
Imperative Style

Because a large program may contain thousands of distinct identifiers, symbol tables must permit **efficient lookup**.

- **How?**
- **Hash Table**
 - e.g., $\sigma_2 = \sigma_1 + \{a \mapsto \text{int}\}$ means insert $\langle a, \text{int} \rangle$ in the hash table

Symbol tables need to **support deletion** easily.

- **How?**
- **Hash Table with external chaining**
 - e.g., $\text{hash}(a) \rightarrow \langle a, \text{int} \rangle \rightarrow \langle a, \text{string} \rangle$



Efficient Imperative Symbol Tables

```
struct bucket { string key; void  
*binding; struct bucket *next; };
```

```
#define SIZE 109
```

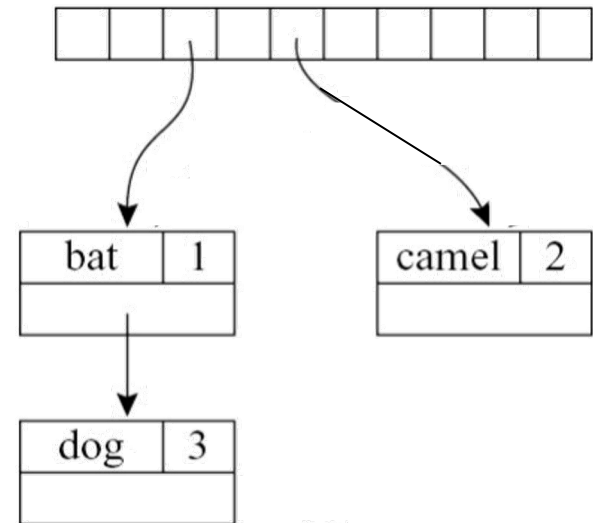
```
struct bucket *table[SIZE];
```

```
unsigned int hash(char *s0)  
{ unsigned int h=0; char *s;  
  for(s=s0; *s; s++)  
    h=h*65599 + *s;  
  return h;  
}
```

```
struct bucket *Bucket (string key,  
void *binding, struct bucket *next) {  
  struct bucket  
*b=checked_malloc(sizeof(*b));  
  b->key = key; b->binding = binding;  
  b->next = next;  
  return b;  
}
```

- **Hash table**: an array of buckets

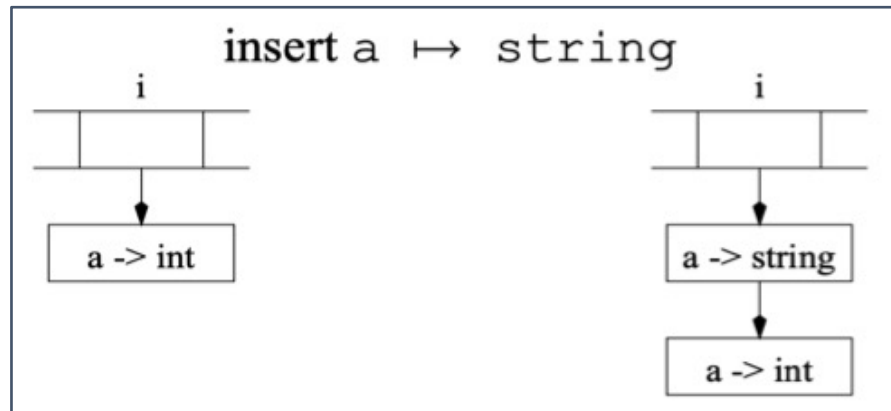
$$h = (\alpha^{n-1}c_1 + \alpha^{n-2}c_2 + \dots + \alpha c_{n-1} + c_n)$$



Effective Imperative Symbol Tables

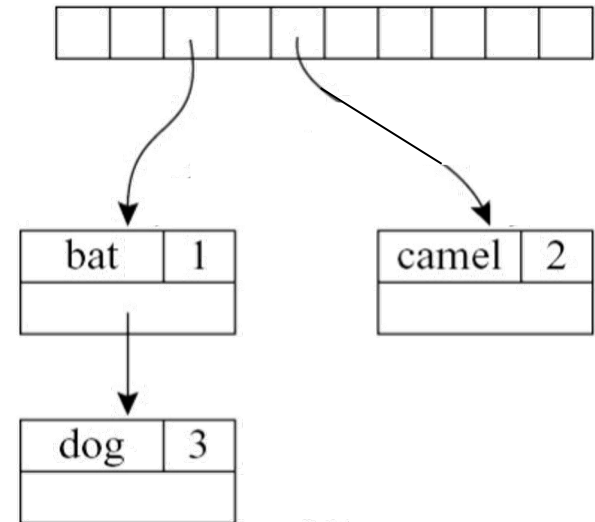
```
void insert(string key, void *binding) {  
    int index=hash(key)%SIZE;  
    table[index]=Bucket(key, binding, table[index]);  
}
```

- This function inserts a new element at the front of a bucket
 - Consider $\sigma + \{a \mapsto \text{string}\}$ when σ contains $a \mapsto \text{int}$ already
 - This function leaves $a \mapsto \text{int}$ in the bucket and puts $a \mapsto \text{string}$ earlier in the list.
 - e.g. $\text{hash}(a) \rightarrow \langle a, \text{string} \rangle \rightarrow \langle a, \text{int} \rangle$



Effective Imperative Symbol Tables

```
void *lookup(string key) {  
    int index=hash(key)%SIZE  
    struct bucket *b;  
    for (b = table[index]; b; b=b->next)  
        if (0==strcmp(b->key, key))  
            return b->binding;  
    return NULL;  
}
```



- This function looks up the entry for the identifier *key* in the symbol table
 - Calculate the array *index* for *key* via the *hash* function
 - Iterate the buckets in *table[index]* to find the findings
- E.g., to lookup *dog*

Effective Imperative Symbol Tables

```
void pop(string key) {  
    int index=hash(key)%SIZE  
    table[index]=table[index].next;  
}
```

- This function pops the first bucket in the linked list of *key*
 - Consider $\sigma + \{a \mapsto \text{string}\}$ when σ contains $a \mapsto \text{int}$ already.
 - When $\text{pop}(a)$ is done at the end of a 's scope, σ is restored.
 - E.g., $\text{hash}(a) \rightarrow \langle a, \text{string} \rangle \rightarrow \langle a, \text{int} \rangle \Rightarrow \text{hash}(a) \rightarrow \langle a, \text{int} \rangle$
- Insertion and pop work in a stack-like fashion



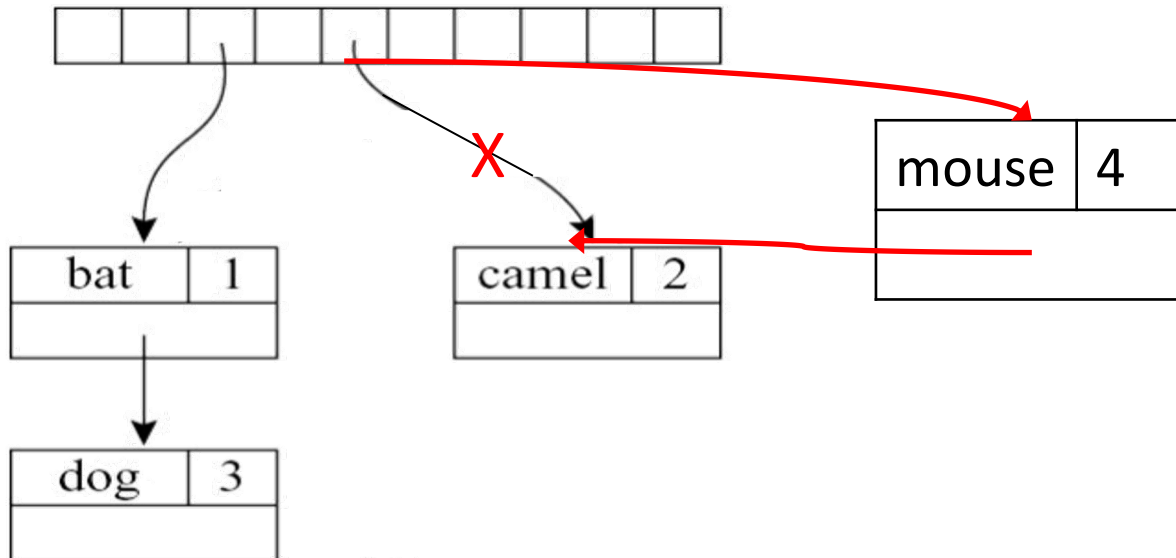
Efficient Functional Symbol Tables

Functional Style

- We wish to compute $\sigma' = \sigma + \{a \mapsto \tau\}$ in such a way that we still have σ available to look up identifiers.
 - do not change σ
- We create a new table σ' , instead of modifying σ
- Can we achieve this with hash tables?

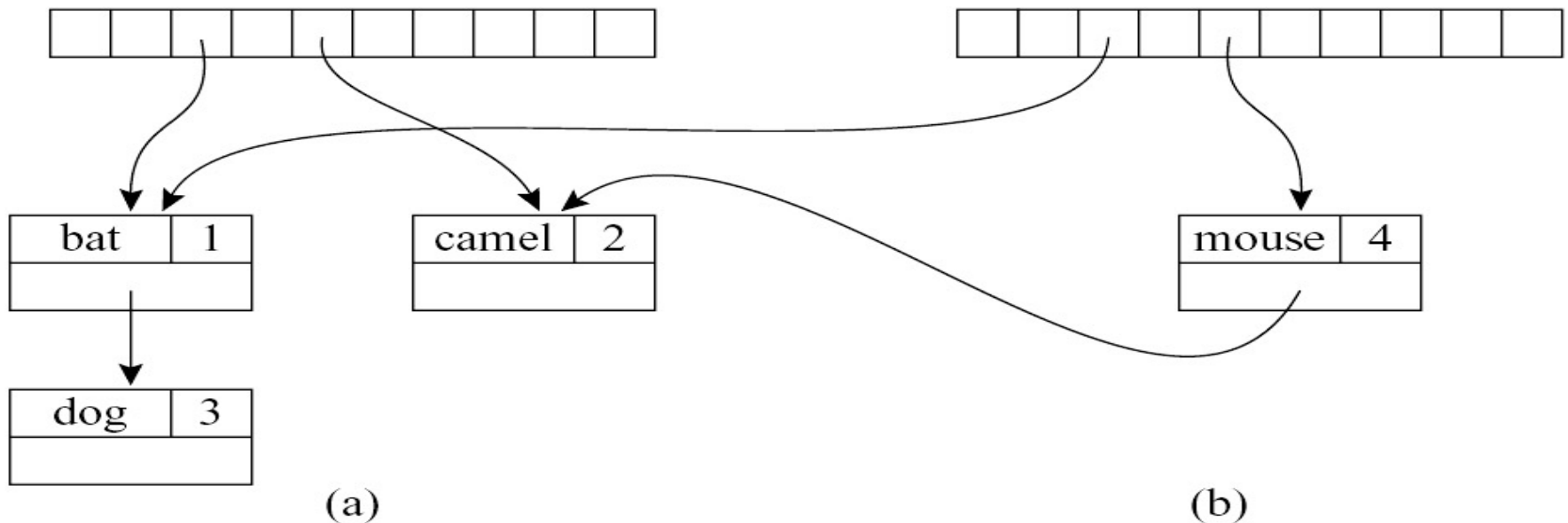
Efficient Functional Symbol Tables

- $m2 = m1 + \{\text{mouse} \mapsto 4\}$
- $m1 = \{\text{bat} \mapsto 1, \text{camel} \mapsto 2, \text{dog} \mapsto 3\}$, suppose $\text{index}(\text{camel}) = \text{index}(\text{mouse}) = 5$
- $\text{Hash}(\text{mouse}) \rightarrow \langle \text{mouse}, 4 \rangle$: no longer have $m1$



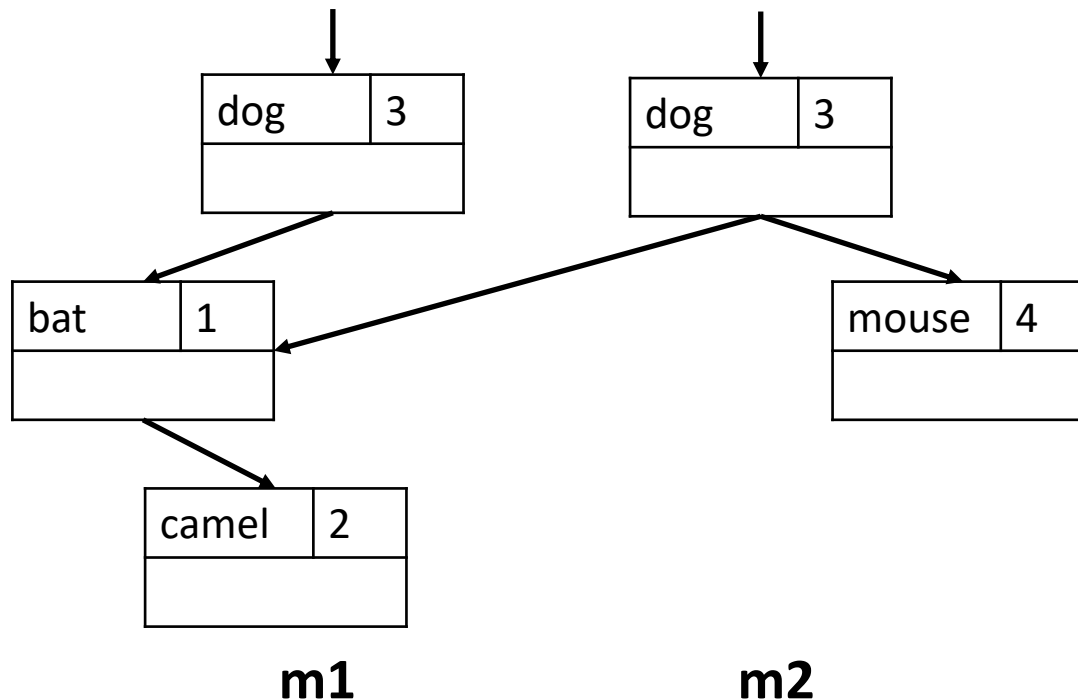
Efficient Functional Symbol Tables

- $m2 = m1 + \{\text{mouse} \mapsto 4\}$
- $m1 = \{\text{bat} \mapsto 1, \text{camel} \mapsto 2, \text{dog} \mapsto 3\}$, suppose $\text{index}(\text{camel}) = \text{index}(\text{mouse}) = 5$
- $\text{Hash}(\text{mouse}) \rightarrow \langle \text{mouse}, 4 \rangle$: no longer have $m1$
- Copy the array, but share all the old buckets: **not efficient**
 - The array in a hash table should be quite large



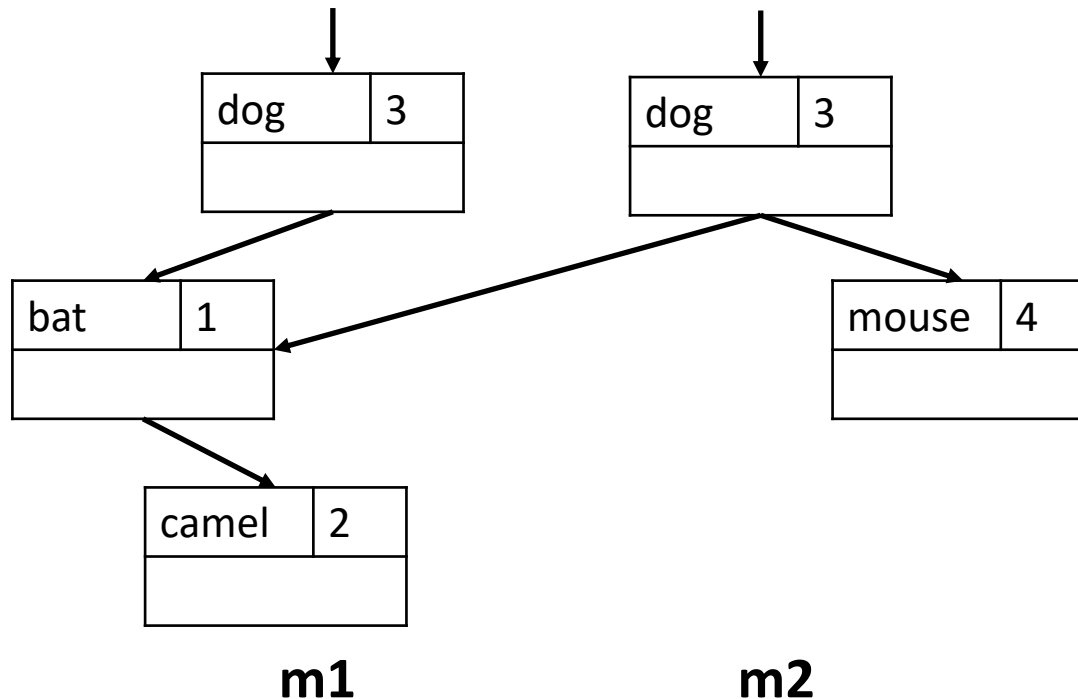
Efficient Functional Symbol Table

- We can perform such “functional” additions to search trees efficiently by using **binary search trees**.
 - Each node contains a binding
 - Use string comparison for ordering



Efficient Functional Symbol Table

- **Insert**: Copy the nodes from the root to the parent of the inserted node
 - e.g., insert `<mouse, 4>`
 - `m1` keeps unchanged. *We don't need to copy the whole tree.*
- **Lookup**: $O(\log(n))$ for a balanced tree of n nodes



Summary: Implementation of Symbol Tables

- **Imperative Style**

- After entering a scope, bindings are added to the table using side effects. (The old table is destroyed)
- When exiting a scope, auxiliary info is used to remove previous additions. (Old table reconstructed)

- **Functional Style**

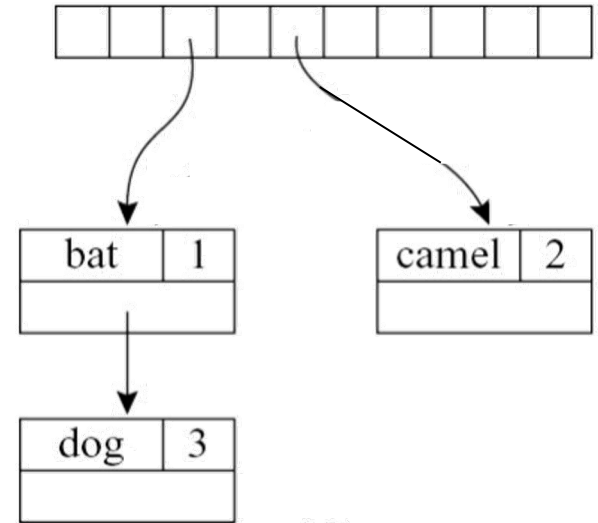
- After entering a scope, a new environment is created by adding to the old one. The old table remains intact.
- When exiting a scope, the old table is retrieved.

Outline

- Symbol Tables
- **Symbols in the Tiger Compiler**
- Type Checking

Issue with Table Implementations

```
void *lookup(string key) {  
    int index=hash(key)%SIZE  
    struct bucket *b;  
    for (b = table[index]; b; b=b->next)  
        if (0==strcmp(b->key, key))  
            return b->binding;  
    return NULL;  
}
```



- **Problem:** We need **expensive string comparisons** when performing a lookup.
- **Solution:** Convert each string to a **symbol**
 - Each **symbol** object is associated with an integer value
 - All occurrences of the same string map onto the same **symbol** (different strings map onto different symbols)

Important Properties of Symbol

- Extracting an integer **hash-key** is very fast: use the Symbol-pointer itself as the integer hash-key. (for hash tables)
- Comparing symbols for **equality** is fast (just pointer or integer comparison).
- Comparing two symbols for “**greater-than**” (in some arbitrary ordering) is very fast (for binary search trees)

Symbols in The Tiger Compiler

The interface of symbols and symbol tables

- **void ***: We want **different notions of binding** for different purposes in the compiler – type bindings for types, value bindings for variables and functions

```
typedef struct S_symbol_ *S_symbol;  
S_symbol S_symbol (string);           /* Symbol */  
string S_name(S_symbol);  
  
typedef struct TAB_table_ *S_table;  
S_table S_empty( void);  
void S_enter( S_table t, S_symbol sym, void *value);  
void *S_look( S_table t, S_symbol sym);  
void S_beginScope( S_table t);        /* Symbol Table */  
void S_endScope( S_table t);
```

The Implementation of Symbols

```
static S_symbol mksymbol (string name , S_symbol next) {  
    S_symbol s = checked_malloc(sizeof(*s));  
    s->name = name; s->next = next;  
    return s;  
}
```

```
S_symbol S_symbol (string name) {  
    int index = hash(name)%SIZE;  
    S_symbol syms = hashtable[index], sym;  
    for ( sym = syms; sym; sym = sym->next)  
        if (0 == strcmp(sym->name, name)) return sym;  
    sym = mksymbol(name, syms);  
    hashtable[index] = sym;  
    return sym;  
}
```

```
string S_name (S_symbol sym) {  
    return sym->name;  
}
```

The Implementation of Symbol Tables

- The Tiger compiler in C uses destructive-update environments.
- An imperative table is implemented using a hash table.
- TAB: a generic hash table module

```
// make a new S_Table
S_table S_empty(void) {
    return TAB_empty();
}

// insert a binding
void S_enter(S_table t, S_symbol sym, void *value){
    TAB_enter(t,sym,value);
}

// look up a symbol
void *S_look(S_table t, S_symbol sym) {
    return TAB_look(t,sym);
}
```

The Implementation of Symbol Tables

For destructive-update environments:

- **S_beginScope**: Remembers the current state of the table
- **S_endScope**: Restores the table to where it was at the most recent beginScope that has not already been ended.

```
static struct S_symbol_ marksym = { "<mark>", 0 };

void S_beginScope ( S_table t) {
    S_enter(t, &marksym, NULL);
}

void S_endScope( S_table t) {
    S_symbol s;
    do
        s= TAB_pop(t);
    while (s != &marksym);
}
```

The Implementation of Symbol Tables

How can we exactly pop those bindings after `enterScope`?

Auxiliary stack:

- Showing in what order the symbols were “pushed” into the symbol table.
- As each symbol in this stack is popped, the head binding in its bucket is removed.
- **beginScope**: pushes a special marker onto the stack
- **endScope**: pops symbols off the stack until finds the topmost marker.

The Implementation of Symbol Tables

- The auxiliary stack can be integrated into the Binder by having a global variable `top` showing the **most recent Symbol** bound in the table.
- Pushing: copy `top` into the `prevtop` field of the Binder.

```
struct TAB_table_ {  
    binder table[TABSIZE];  
    void *top;  
};
```

```
t->table[index] = Binder(key, value, t->table[index], t->top);
```

```
static binder Binder(void *key, void *value, binder next, void  
*prevtop) {  
    binder b = checked_malloc(sizeof(*b));  
    b->key = key; b->value=value; b->next=next;  
    b->prevtop = prevtop;  
    return b;  
}
```